Taller No. 2 Jakarta EE + Tier de Negocio

Diseño de la arquitectura y evidencias de funcionamiento
Módulo de gestión de solicitudes de recursos

Nombre del estudiante
Arquitectura de Software – Departamento de Ingeniería de Sistemas
Pontificia Universidad Javeriana

2026-30

Resumen

Se implementó el tier de negocio de un módulo que administra las solicitudes de recursos durante emergencias. Las reglas de negocio viven en un EJB de sesión `@Stateless`, que solo conoce una interfaz de repositorio; el tier de datos la implementa con una colección en memoria. El documento presenta la arquitectura, explica cómo se desacopló el negocio de la persistencia y muestra las evidencias: 13 pruebas unitarias del EJB sin servidor y 13 peticiones REST con 24 aserciones sobre Payara 7. Todas pasan.

Índice

1. Arquitectura propuesta **2**
1.1. Contrato REST . . . 3
1.2. Máquina de estados . . . 4

2. Desacoplamiento entre el tier de negocio y el tier de datos **4**
2.1. Decisiones y supuestos . . . 5

3. Cobertura de las reglas de negocio **6**

4. Pruebas unitarias del EJB **6**
4.1. Cómo se prueba el EJB sin servidor . . . 6
4.2. Resultados . . . 7

5. Pruebas sobre Payara con Postman **7**
5.1. Registrar una solicitud . . . 8
5.2. Validaciones del negocio . . . 9
5.3. Consultar . . . 9
5.4. Atender una solicitud . . . 11
5.5. Consultar pendientes . . . 12
5.6. Ejecución completa de la colección . . . 13

6. Conclusión **14**

A. Vista dinámica **15**

1. Arquitectura propuesta

La solución es una aplicación Jakarta EE 11 empaquetada como un único WAR (`negocio-jee.war`) y desplegada en Payara 7. Internamente se organiza en tres capas lógicas más un modelo de dominio compartido:

- **Presentación (REST).** Endpoints JAX-RS delgados que solo traducen HTTP/JSON a llamadas al negocio. Existen para evidenciar el funcionamiento; no contienen reglas.
- **Negocio (EJB).** El EJB de sesión sin estado `GestionSolicitudesBean` encapsula todas las reglas. Se expone por la interfaz local `GestionSolicitudes`.
- **Datos (en memoria).** `SolicitudRepositoryMemoria`, un EJB `@Singleton` que guarda las solicitudes en un `ConcurrentHashMap`.

La regla que gobierna el diseño es que **las dependencias siempre apuntan hacia el tier de negocio**: la presentación depende del negocio y los datos dependen del negocio; el negocio no depende de ninguno de los dos.

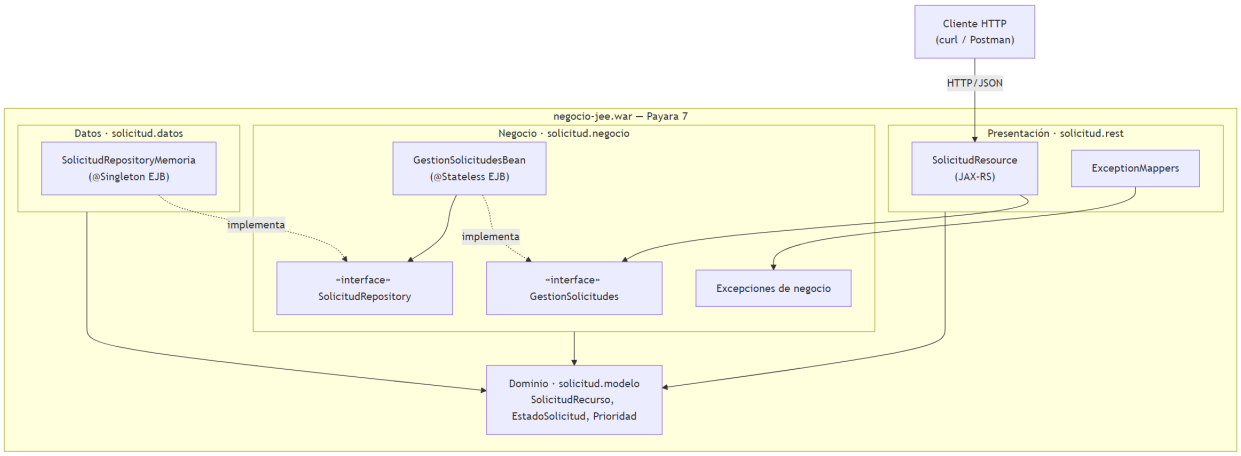


Figura 1: Capas y colaboración entre componentes.

Tabla 1: Paquetes de `edu.javeriana.negocio.jakarta` y su tier.

Paquete	Contenido	Tier
`solicitud.rest`	`SolicitudResource` y tres `ExceptionMapper`	Presentación
`solicitud.rest.dto`	`SolicitudEntrada`, `SolicitudSalida`, `ErrorSalida`	Presentación
`solicitud.negocio`	`GestionSolicitudes` (`@Local`), `GestionSolicitudesBean` (`@Stateless`), `SolicitudRepository`, excepciones	Negocio
`solicitud.modelo`	`SolicitudRecurso`, `EstadoSolicitud`, `Prioridad`	Dominio
`solicitud.datos`	`SolicitudRepositoryMemoria` (`@Singleton`)	Datos
`hello`	`HelloApplication`: activa JAX-RS en `/rest`	Configuración

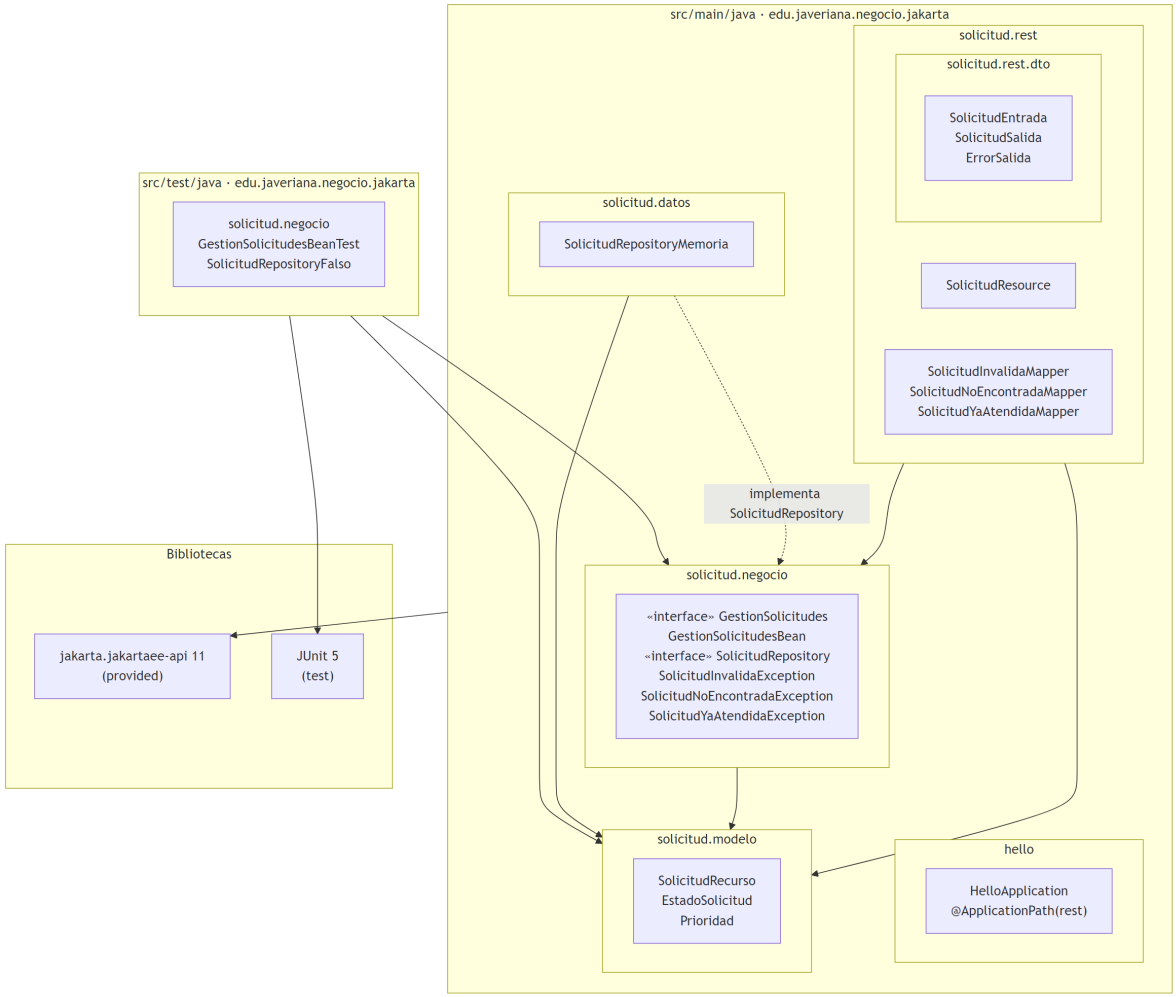


Figura 2: Diagrama de paquetes. Las flechas punteadas indican que un paquete implementa una interfaz definida en otro.

1.1. Contrato REST

Ruta base: `http://localhost:8080/negocio-jee/rest`. Los DTO son el contrato JSON y viven solo en la capa REST: `SolicitudEntrada` no tiene `id` ni `estado`, de modo que el cliente no puede fijarlos.

Tabla 2: Operaciones expuestas.

Operación	**Método y ruta**	**Respuestas**
Registrar solicitud	`POST /solicitudes`	201 ů 400
Consultar todas	`GET /solicitudes`	200
Buscar por identificador	`GET /solicitudes/{id}`	200 ů 404
Consultar pendientes	`GET /solicitudes/pendientes`	200
Atender solicitud	`PUT /solicitudes/{id}/atender`	200 ů 404 ů 409

1.2. Máquina de estados

Una solicitud tiene una única transición válida, de `PENDIENTE` a `ATENDIDA`. La controla el EJB: es el único componente que asigna el estado.

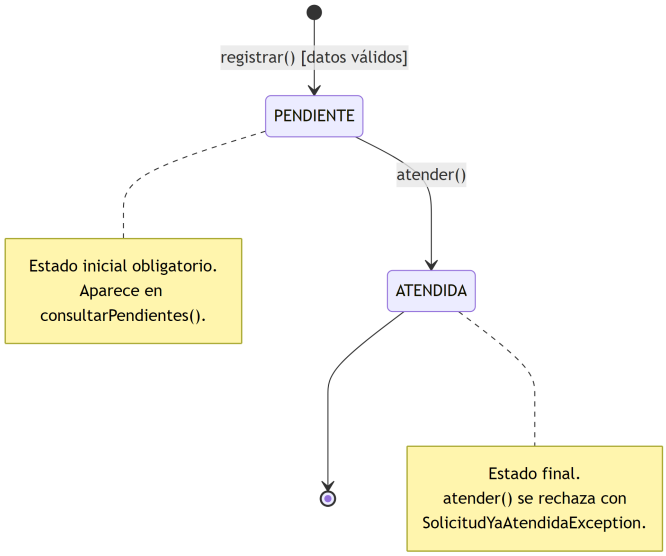


Figura 3: Máquina de estados de la solicitud.

2. Desacoplamiento entre el tier de negocio y el tier de datos

El EJB de negocio no sabe cómo ni dónde se guardan las solicitudes. Esto se logra con cinco decisiones:

1. **Inversión de dependencias.** La interfaz `SolicitudRepository` pertenece al paquete *de negocio* y expresa lo que el negocio necesita de la persistencia (guardar, actualizar, buscar, listar). El tier de datos la implementa. Por eso `solicitud.negocio` no importa ninguna clase de `solicitud.datos`: la dependencia va de datos hacia negocio y no al revés.

2. **Inyección por el contenedor.** El EJB declara `@Inject SolicitudRepository` y el contenedor le entrega la implementación disponible. Nunca hace `new SolicitudRepositoryMemoria()`.

3. **Sustituibilidad.** Para pasar a una base de datos basta con escribir otra clase, por ejemplo `SolicitudRepositoryJpa`, que implemente la misma interfaz. Ni el EJB ni la capa REST cambian.

4. **Repositorio sin reglas.** `SolicitudRepositoryMemoria` solo almacena, recupera y genera el identificador. No valida ni cambia estados.

5. **Negocio que se prueba sin servidor.** Como el EJB solo depende de una interfaz, las pruebas JUnit lo crean con `new` y un repositorio falso. Que todas las reglas se cumplan así es la evidencia de que el negocio no está atado a la persistencia.

```java
// solicitud.negocio: la interfaz la define el negocio, no el tier de datos
public interface SolicitudRepository {
    SolicitudRecurso guardar(SolicitudRecurso solicitud);
    SolicitudRecurso actualizar(SolicitudRecurso solicitud);
    Optional<SolicitudRecurso> buscarPorId(long id);
    List<SolicitudRecurso> listarTodas();
}

// solicitud.negocio: el EJB solo conoce la interfaz
@Stateless
public class GestionSolicitudesBean implements GestionSolicitudes {
    @Inject
    private SolicitudRepository repositorio;
    ...
}

// solicitud.datos: el tier de datos implementa la interfaz del negocio
@Singleton
public class SolicitudRepositoryMemoria implements SolicitudRepository { ... }
```

El mismo principio se aplica hacia arriba: la capa REST depende de la interfaz `GestionSolicitudes`, no de la clase del EJB.

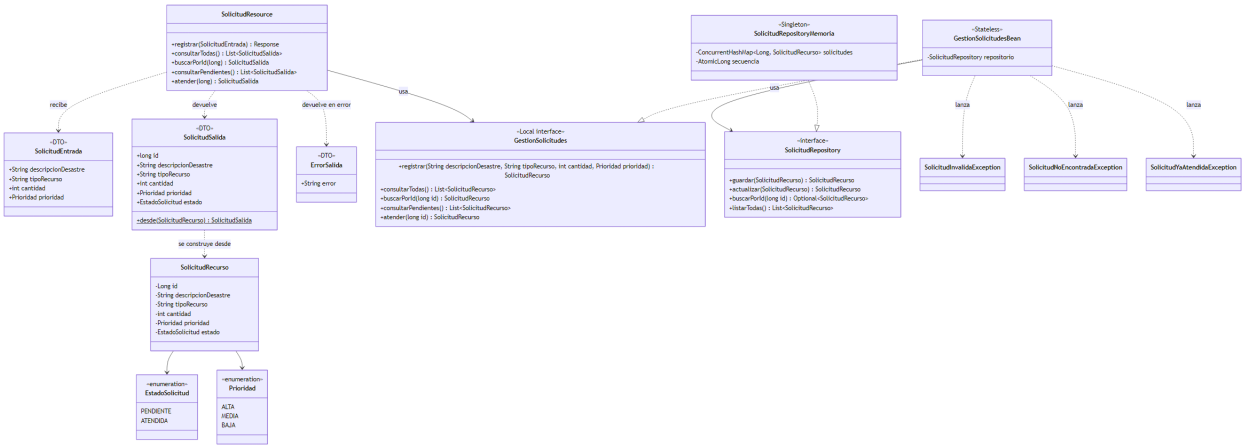


Figura 4: Diagrama de clases.

2.1. Decisiones y supuestos

- **`@Stateless` para el negocio:** las operaciones no guardan estado conversacional, así que el contenedor puede reutilizar instancias.
- **`@Singleton` para el almacén:** la colección debe ser única y compartida por todas las instancias del EJB de negocio; además se usan `ConcurrentHashMap` y `AtomicLong`.
- **Interfaz `@Local`:** el único cliente (la capa REST) está en la misma JVM; no se necesita una interfaz `@Remote`.
- **Excepciones de aplicación:** las excepciones de negocio llevan `@ApplicationException`, así el contenedor no las envuelve en `EJBException` y la capa REST las traduce a 400, 404 o 409.

- **Prioridad:** el enunciado no define sus valores; se modela como enumerado `ALTA`, `MEDIA`, `BAJA` y se trata como dato obligatorio.
- **Sin persistencia duradera:** los datos se pierden al reiniciar el servidor, lo que el enunciado admite al pedir una colección en memoria.

3. Cobertura de las reglas de negocio

Todas las validaciones se ejecutan en `GestionSolicitudesBean`. La Tabla 3 relaciona cada regla del enunciado con la prueba unitaria y la petición de Postman que la verifican.

Tabla 3: Reglas de negocio y pruebas que las verifican.

Regla	Prueba JUnit	Postman	HTTP
La cantidad debe ser mayor que cero	`rechazarCantidadCeroONegativa`	03	400
La descripción del desastre es obligatoria	`rechazarSinDescripcion`	04	400
El tipo de recurso es obligatorio	`rechazarSinTipoRecurso`	05	400
La prioridad es obligatoria (supuesto)	`rechazarSinPrioridad`	06	400
Toda solicitud nueva inicia en `PENDIENTE`	`registrarSolicitudValida`	01	201
No se atiende una solicitud inexistente	`atenderInexistente`	12	404
No se atiende dos veces la misma solicitud	`atenderDosVeces`	11	409
Las validaciones ocurren en el negocio	todas (sin servidor ni REST)	—	—

La última regla queda demostrada por la forma de las pruebas unitarias: crean el EJB con `new` y un repositorio falso, sin capa REST ni servidor, y aun así todas las validaciones se cumplen.

4. Pruebas unitarias del EJB

4.1. Cómo se prueba el EJB sin servidor

```
class GestionSolicitudesBeanTest {

    private GestionSolicitudesBean gestion;

    @BeforeEach
    void prepararEjb() {
        gestion = new GestionSolicitudesBean(new SolicitudRepositoryFalso());
    }

    @Test
    @DisplayName("No se puede atender dos veces la misma solicitud")
    void atenderDosVeces() {
        SolicitudRecurso creada = registrarValida();
        gestion.atender(creada.getId());

        assertThrows(SolicitudYaAtendidaException.class,
                     () -> gestion.atender(creada.getId()));
    }
}
```

4.2. Resultados

Tabla 4: Pruebas unitarias de `GestionSolicitudesBean`.

#	Método	Qué verifica	Resultado
1	`registrarSolicitudValida`	Se registra con id y en estado `PENDIENTE`	**Pasa**
2	`rechazarCantidadCeroONegativa`	Cantidad 0 y -5 se rechazan	**Pasa**
3	`rechazarSinDescripcion`	Descripción nula o en blanco se rechaza	**Pasa**
4	`rechazarSinTipoRecurso`	Tipo de recurso vacío se rechaza	**Pasa**
5	`rechazarSinPrioridad`	Prioridad nula se rechaza	**Pasa**
6	`solicitudInvalidaNoSeGuarda`	Una solicitud rechazada no queda guardada	**Pasa**
7	`consultarTodas`	Devuelve todas las registradas	**Pasa**
8	`buscarPorId`	Devuelve la solicitud pedida	**Pasa**
9	`buscarIdInexistente`	Lanza `SolicitudNoEncontradaException`	**Pasa**
10	`consultarPendientes`	Excluye las atendidas	**Pasa**
11	`atenderPendiente`	Pasa a `ATENDIDA` y queda guardado	**Pasa**
12	`atenderInexistente`	Lanza `SolicitudNoEncontradaException`	**Pasa**
13	`atenderDosVeces`	Lanza `SolicitudYaAtendidaException`	**Pasa**

Salida de `mvn clean package` (extracto; la salida completa está en `evidencias/maven-build.txt`):

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running edu.javeriana.negocio.jakarta.solicitud.negocio.GestionSolicitudesBeanTest
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- war:3.5.1:war (default-war) @ negocio-jee ---
[INFO] Building war: target/negocio-jee.war
[INFO] BUILD SUCCESS
```

5. Pruebas sobre Payara con Postman

La colección `postman/negocio-jee.postman_collection.json` contiene 13 peticiones que se ejecutan en orden sobre un servidor recién iniciado. Cada una tiene aserciones automáticas que comprueban el código HTTP y el contenido de la respuesta.

Tabla 5: Peticiones de la colección y resultado obtenido sobre Payara 7.

#	Caso	Petición	Esperado	Obtenido
01	Registrar solicitud válida	`POST /solicitudes`	201	201
02	Registrar segunda solicitud	`POST /solicitudes`	201	201
03	Cantidad cero	`POST /solicitudes`	400	400
04	Descripción vacía	`POST /solicitudes`	400	400
05	Tipo de recurso vacío	`POST /solicitudes`	400	400
06	Sin prioridad	`POST /solicitudes`	400	400
07	Consultar todas	`GET /solicitudes`	200	200
08	Buscar por id	`GET /solicitudes/1`	200	200
09	Buscar id inexistente	`GET /solicitudes/999`	404	404
10	Atender solicitud pendiente	`PUT /solicitudes/1/atender`	200	200
11	Atender dos veces	`PUT /solicitudes/1/atender`	409	409
12	Atender inexistente	`PUT /solicitudes/999/atender`	404	404
13	Consultar pendientes	`GET /solicitudes/pendientes`	200	200

5.1. Registrar una solicitud

El cliente no envía `id` ni `estado`: el repositorio asigna el identificador y el EJB fija el estado `PENDIENTE`.

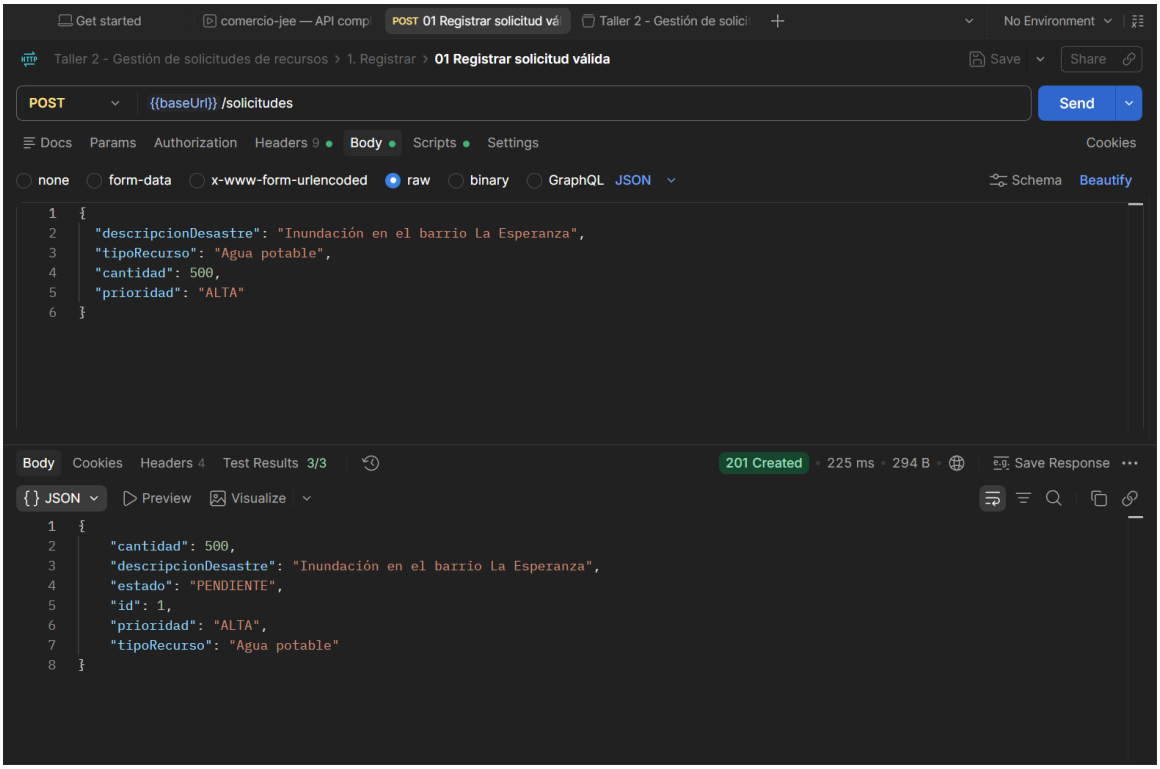


Figura 5: Petición 01: registro de una solicitud válida (201 Created). Arriba el cuerpo enviado; abajo la respuesta con `id` 1 y estado `PENDIENTE`.

5.2. Validaciones del negocio

Cuando el EJB rechaza los datos, lanza `SolicitudInvalidaException` y un `ExceptionMapper` la convierte en un 400 con el mensaje de la regla. Las peticiones 03 a 06 prueban cada validación:

```
04  descripcionDesastre: ""  ->  400 { "error": "La descripción del desastre es obligatoria" }
05  tipoRecurso: ""          ->  400 { "error": "El tipo de recurso es obligatorio" }
06  sin prioridad            ->  400 { "error": "La prioridad es obligatoria" }
```

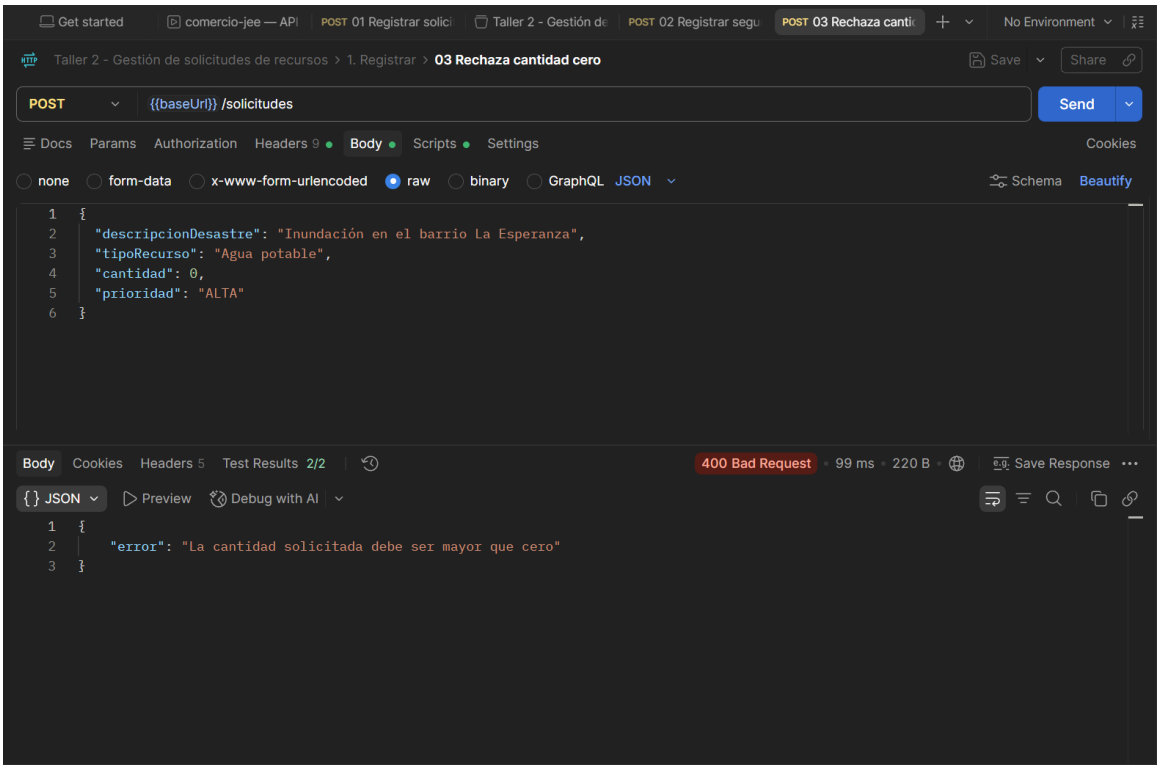


Figura 6: Petición 03: solicitud rechazada por cantidad cero (400 Bad Request).

5.3. Consultar

Solo aparecen las dos solicitudes válidas: las cuatro rechazadas nunca llegaron al repositorio. Buscar un identificador inexistente (petición 09) responde 404 con el mensaje *No existe una solicitud con id 999*.

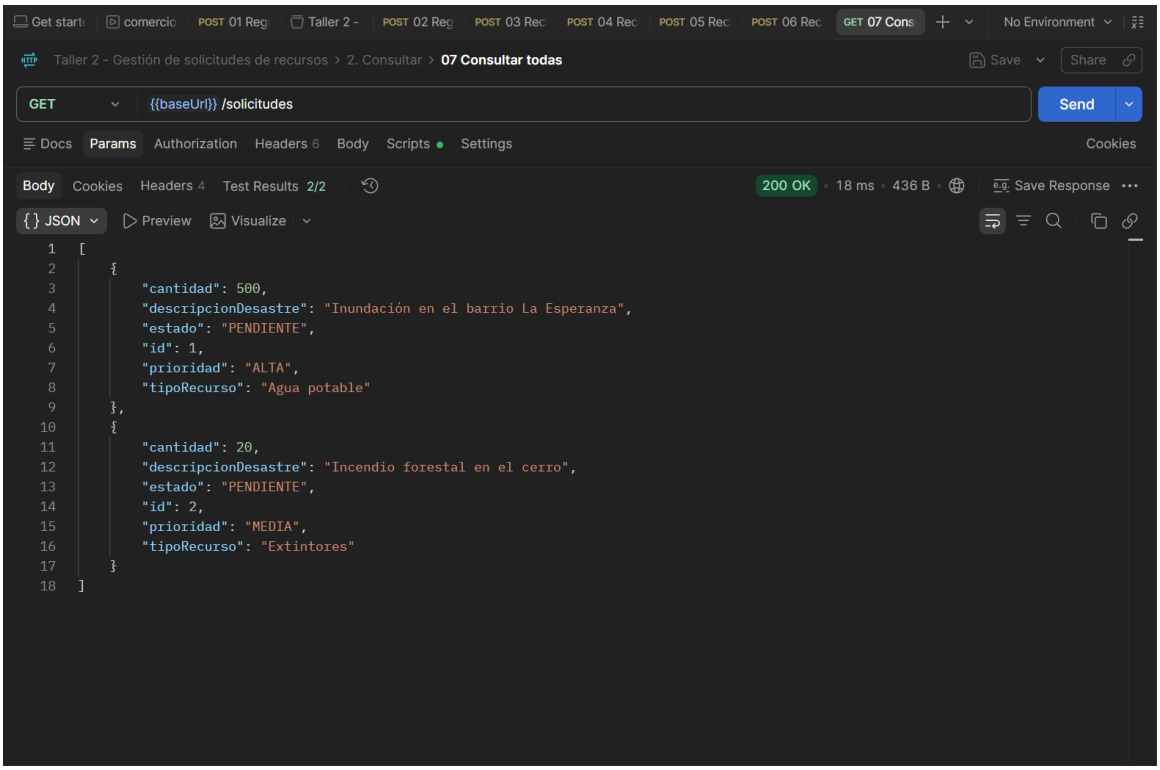


Figura 7: Petición 07: consulta de todas las solicitudes (200 OK); solo están las dos registradas con datos válidos.

5.4. Atender una solicitud

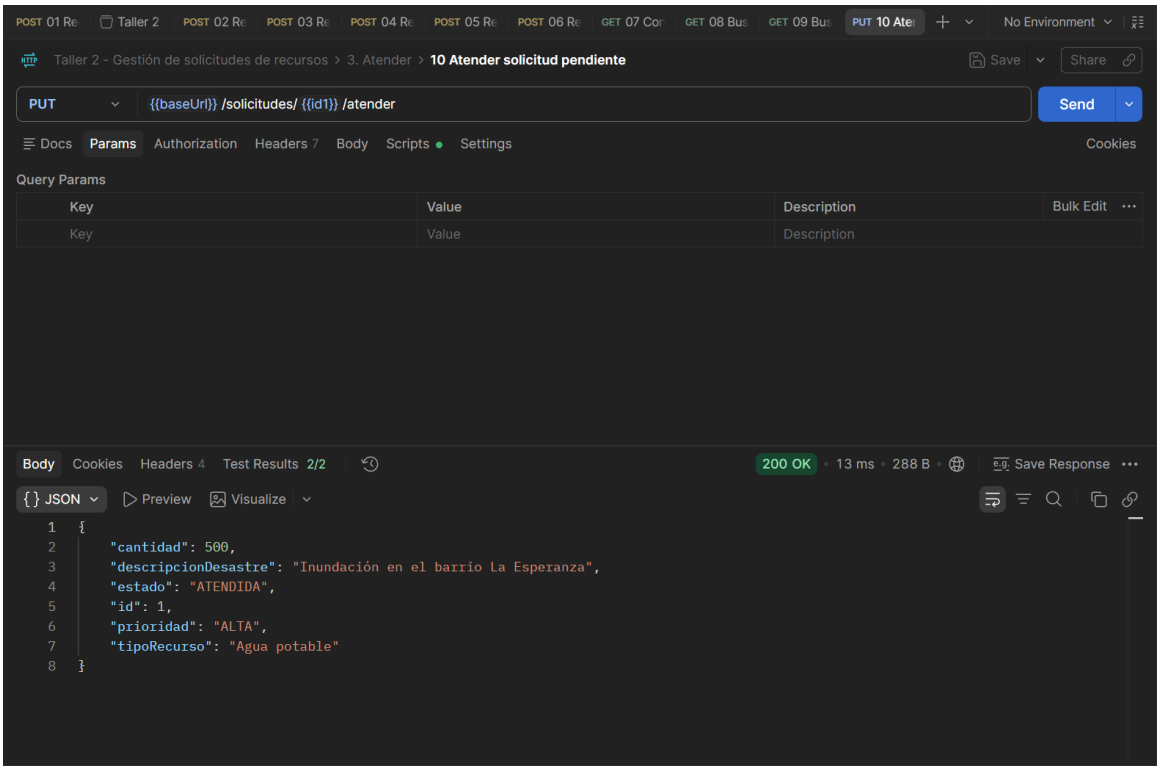


Figura 8: Petición 10: la solicitud 1 pasa de `PENDIENTE` a `ATENDIDA` (200 OK).

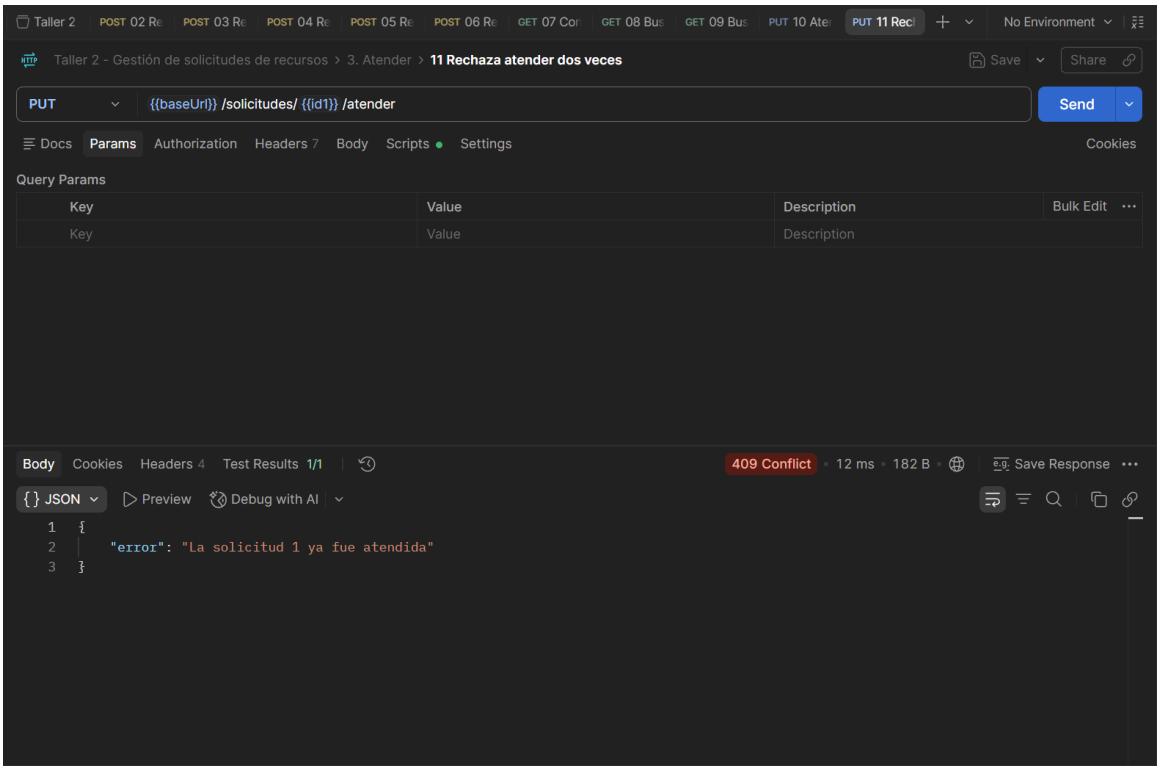


Figura 9: Petición 11: intento de atender de nuevo la solicitud 1 (409 Conflict).

Atender un identificador inexistente (petición 12) responde 404 con el mismo mensaje.

5.5. Consultar pendientes

La solicitud 1 ya no aparece porque fue atendida.

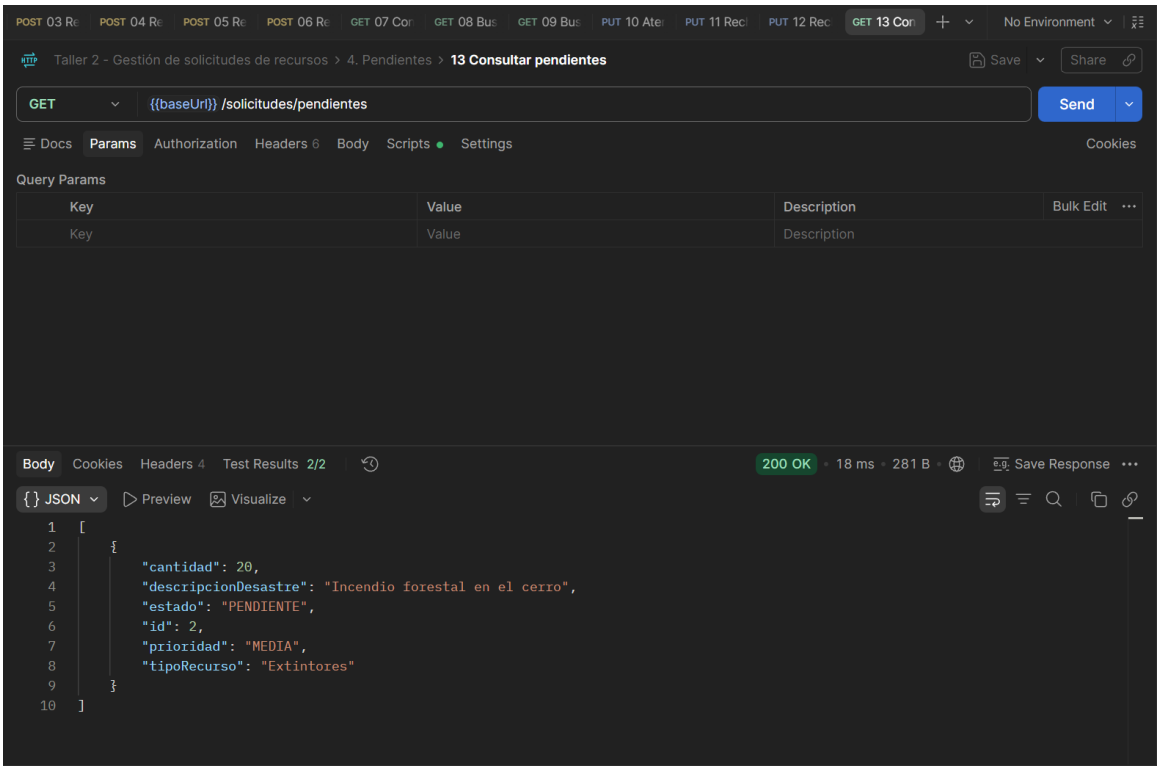


Figura 10: Petición 13: consulta de pendientes (200 OK); solo aparece la solicitud 2.

5.6. Ejecución completa de la colección

Con el servidor reiniciado se ejecutó la colección completa en el Collection Runner de Postman y en Newman, su ejecutor de línea de comandos (`evidencias/newman-salida.txt`):

```
|                         |          executed |            failed |
|               iterations |                 1 |                 0 |
|                 requests |                13 |                 0 |
|            test-scripts |                13 |                 0 |
|              assertions |                24 |                 0 |
```

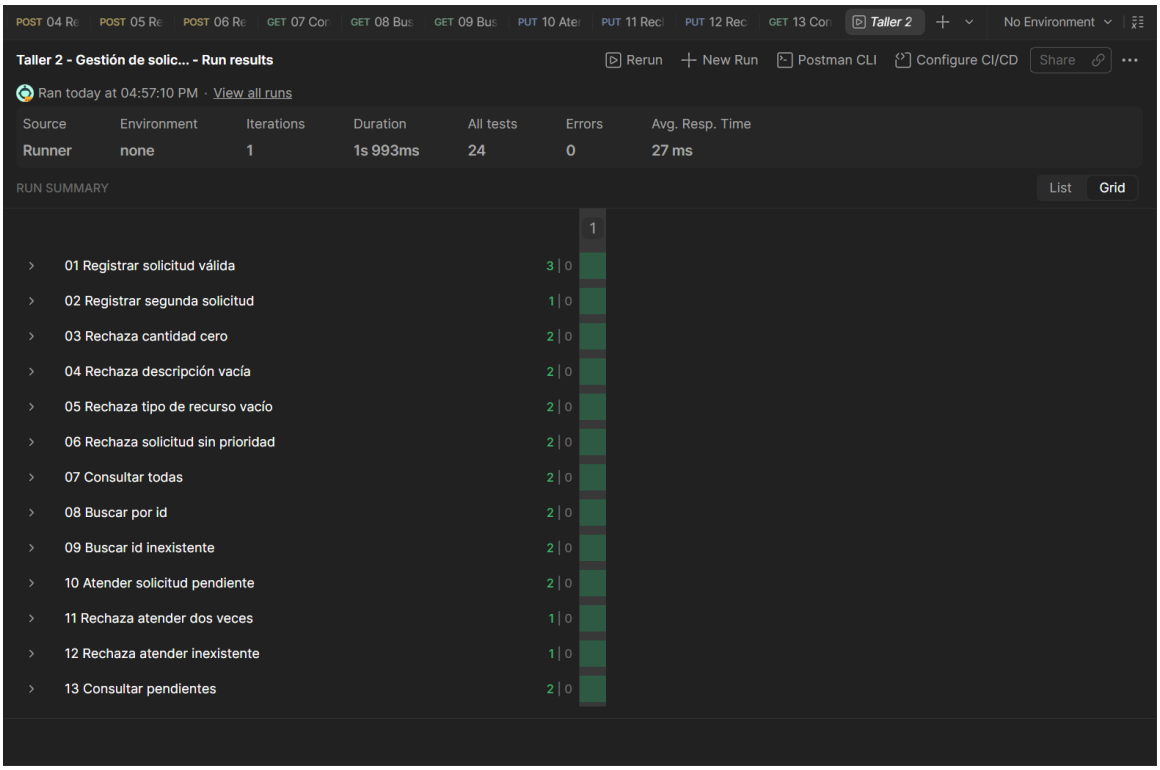


Figura 11: Collection Runner de Postman: 13 peticiones, 24 aserciones aprobadas y 0 errores.

6. Conclusión

Las 13 pruebas unitarias y las 24 aserciones de la colección de Postman pasan. Las cuatro operaciones del enunciado (registrar, consultar todas y por identificador, atender y consultar pendientes) funcionan sobre Payara 7, y cada regla de negocio se verifica en los dos niveles. Como las pruebas unitarias ejecutan el EJB con un repositorio distinto del real y sin servidor, también evidencian que el tier de negocio está desacoplado del tier de datos y que las validaciones no dependen de la interfaz.

Cómo reproducir

```
mvn clean package      # compila y ejecuta las 13 pruebas JUnit
mvn cargo:run          # despliega en Payara 7 (http://localhost:8080)

# Alternativa con Docker
docker build -t negocio-jee .
docker run -p 8080:8080 negocio-jee

# Postman: Import -> postman/negocio-jee.postman_collection.json -> Run collection
# (el servidor debe estar recién iniciado)
```

A. Vista dinámica

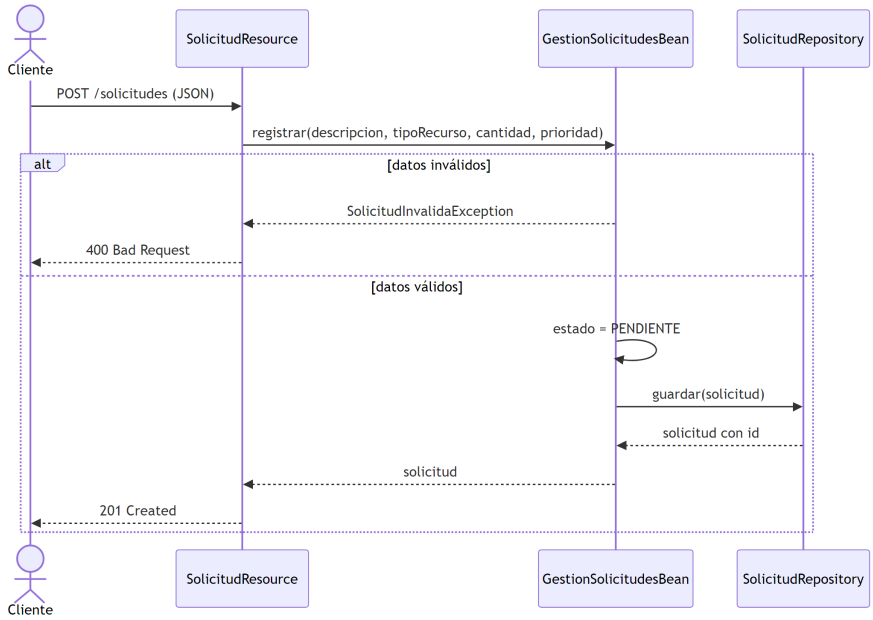


Figura 12: Secuencia: registrar una solicitud.

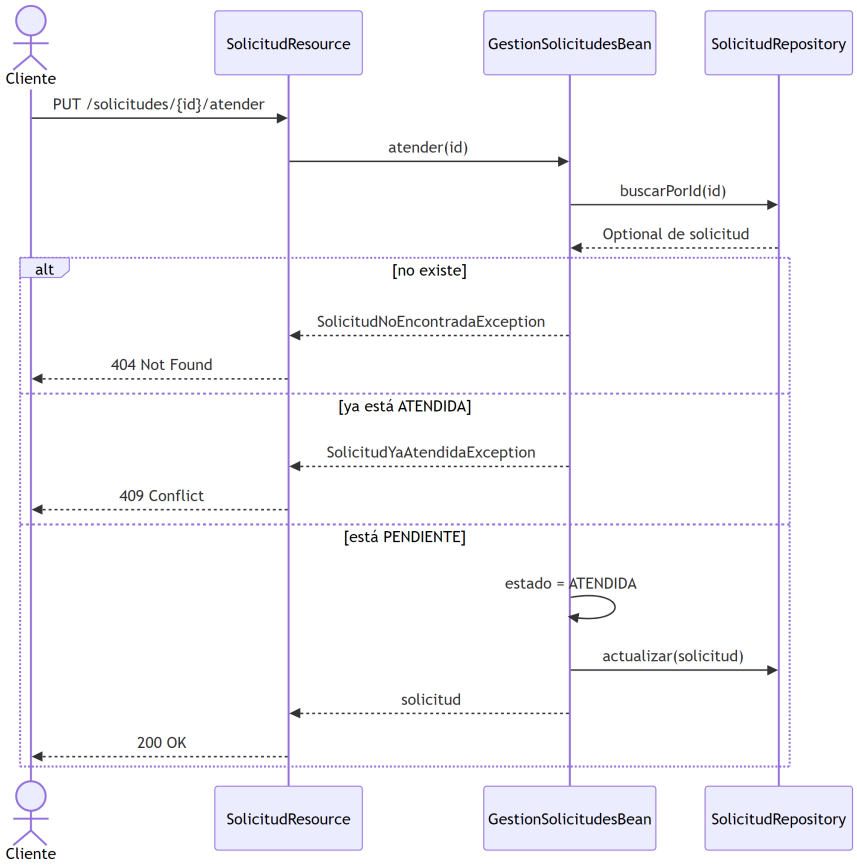


Figura 13: Secuencia: atender una solicitud.